Actividad de Desarrollo 1: Crear un tutorial técnico en CoppeliaSim

Eduardo Roque Viera Santana

Contenido

1. Introducción 3
   a. ¿Qué es CoppeliaSim? 3
   b. ¿Para qué se usa? 3
   c. Objetivo del tutorial 3
2. Creación del entorno 4
   a. Instalación de CoppeliaSim: 4
   b. Creación de escena y añadir primitivas 5
3. Creación de articulaciones 7
   a. ¿Qué es un Joint? 7
   b. Tipos de articulaciones 7
   c. Configuración del motor 8
4. Jerarquía y Cinemática (Padre e Hijo) 11
   a. Qué es el árbol jerárquico 11
   b. Relación padre-hijo 11
   c. Por qué es importante 12
5. Creación del primer eslabón (Brazo 1): 12
   a. Añadir y posicionar la pieza 12
6. Completando el brazo: Codo y Antebrazo 15
   a. Añadir el segundo motor (Codo) 15
   b. Añadir el segundo eslabón (Antebrazo) 17
7. Programación y Control (Scripts) 19
   a. ¿Qué es un script en CoppeliaSim? 19
   b. Diferencia entre scripts Threaded y Non-threaded 19
   c. Creación del script en la base 20
   d. Código de movimiento automático 21
   e. Explicación línea por línea del código 22
   Ajuste de Orientación (Inclinación del Codo) 23
8. Pruebas, Resultados y Resolución de Problemas 24
   a. Resultados de la simulación 24
   b. Problemas encontrados y soluciones aplicadas 24

1. Introducción

a. ¿Qué es CoppeliaSim?

CoppeliaSim es un potente simulador de robótica de propósito general que permite crear, programar y probar sistemas complejos en un entorno 3D realista con físicas integradas. Se distingue por su arquitectura de control distribuida y su alta compatibilidad con múltiples lenguajes de programación, lo que lo convierte en una herramienta estándar tanto para la educación como para la investigación industrial.

b. ¿Para qué se usa?

CoppeliaSim como un banco de pruebas virtual o un "simulador de vuelo", pero para máquinas.

Los usuarios lo usan para crear una copia digital exacta de un robot. El objetivo es ver cómo se comportará esa máquina en la realidad, antes de comenzar a construirla o comprarla.

Básicamente es una máquina de previsión: permite solucionar los errores en la pantalla, es decir, en el programa, para que cuando se instale en el robot real funcione en teoría a la primera.

c. Objetivo del tutorial

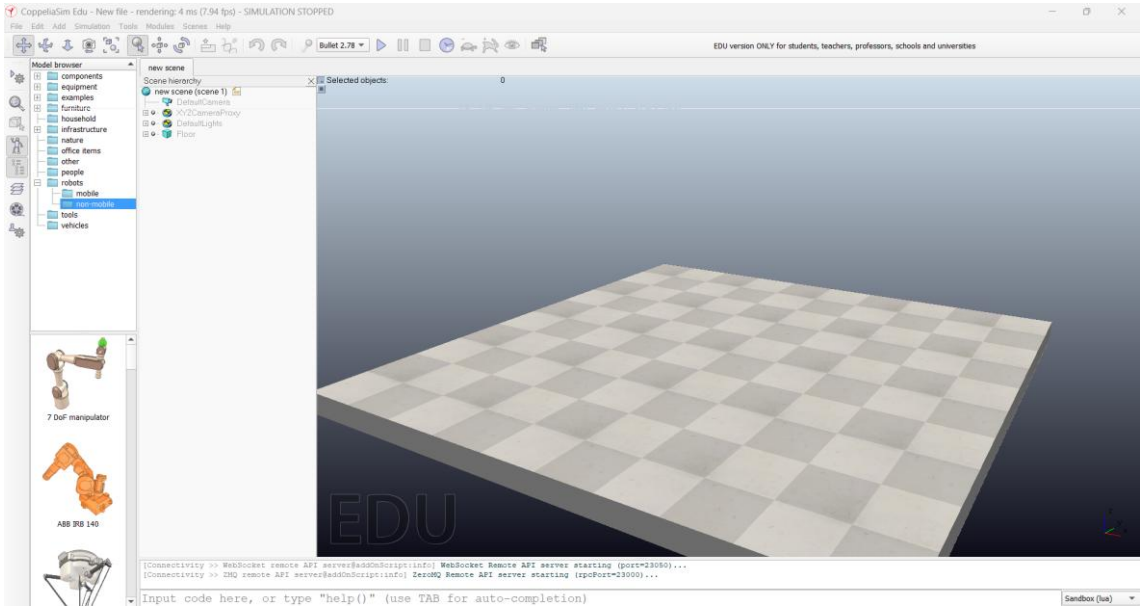
El objetivo principal de este documento es servir como guía técnica. A través de este tutorial, se busca que el lector adquiera las habilidades necesarias para diseñar, ensamblar y programar un brazo robótico básico en el entorno de simulación CoppeliaSim

2. Creación del entorno

a. Instalación de CoppeliaSim:

Para desplegar el entorno de simulación en el equipo, sigue las siguientes instrucciones:

1. **Ejecución del instalador:** Localiza el archivo facilitado (por ejemplo, *CoppeliaSim_Edu_V4_10_0_rev0_Setup.exe* o una versión superior) y ejecútalo haciendo doble clic.
2. **Asistente de configuración:** Sigue los pasos del asistente de instalación, aceptando los términos de licencia y pulsando "Siguiente" hasta finalizar el proceso.
3. **Verificación:** Una vez instalado, abre el programa **CoppeliaSim Edu**. Deberá cargarse la interfaz principal con una escena vacía, lo que confirma que el software está listo para usarse.

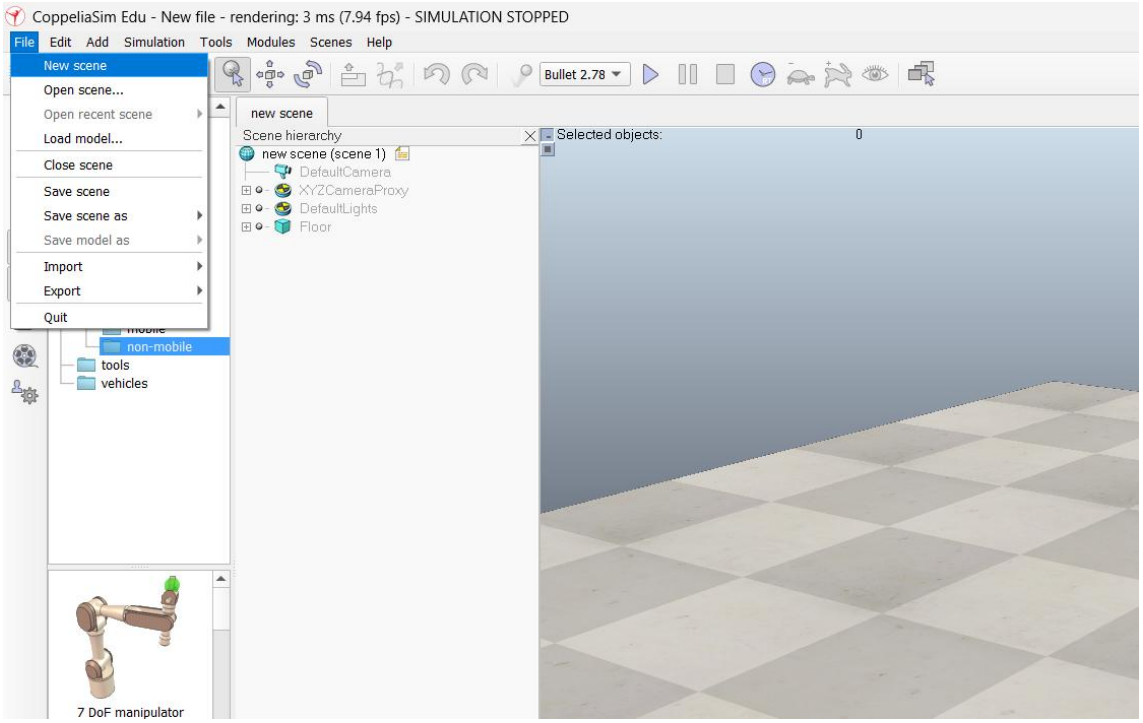


Al iniciar el software, se desplegará la interfaz principal de trabajo mostrando una escena 3D vacía con un suelo en cuadrícula, lista para el diseño. En esta pantalla destacan el **explorador de modelos** a la izquierda (librería de componentes) y la **barra de herramientas** superior para controlar la simulación.

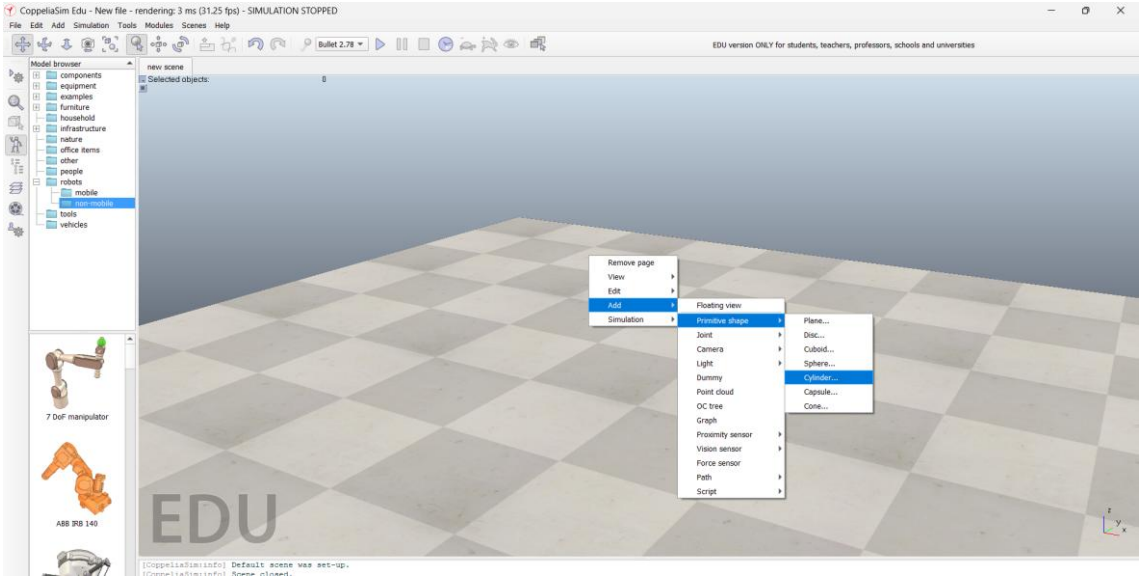
b. Creación de escena y añadir primitivas

Para comenzar el diseño del robot desde cero, seguiremos estos pasos:

1. **Nueva Escena:** Si el área de trabajo no está vacía, accede a File > New Scene en el menú superior.

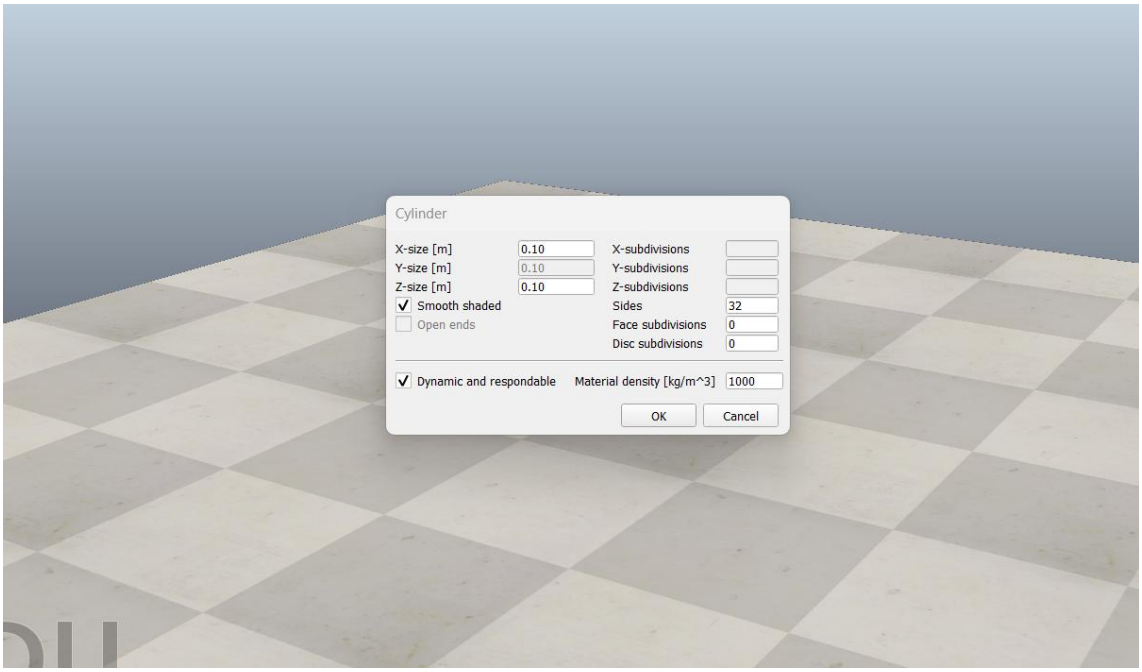


2. **Añadir la Base:** Haremos clic derecho en cualquier parte del visor 3D y seleccionaremos Add > Primitive shape > Cylinder. Esta figura actuará como la **base fija** de nuestro brazo robótico.

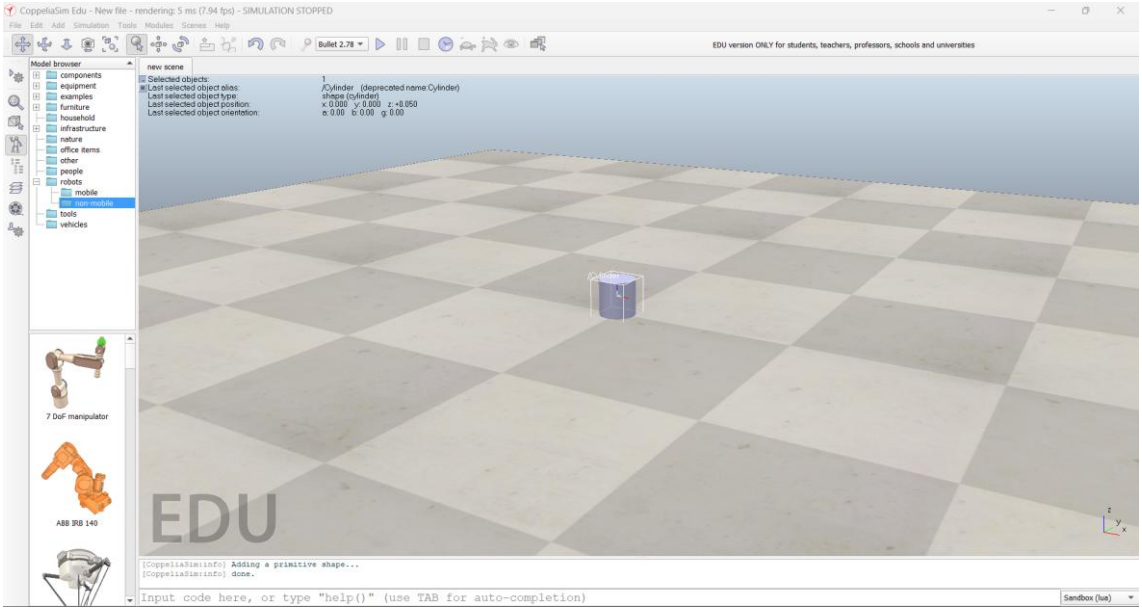


3. **Ajuste de valores:**

- Desmarca la casilla "Keep proportions" (Mantener proporciones) si deseas modificar la altura sin cambiar el ancho.
- Introduce los valores deseados en los campos **X size, Y size y Z size** (Tamaño). Lo dejamos como viene por defecto.



Tras finalizar este paso veremos en la escena algo como esto:



3. Creación de articulaciones

Una vez tenemos la base, necesitamos dotar al robot de movimiento utilizando "Joints".

a. ¿Qué es un Joint?

Un *Joint* (o articulación) es el elemento que vincula dos formas rígidas y define cómo se mueven una respecto a la otra. En la simulación, actúa como el motor o actuador del robot

> *Es el punto de unión entre dos piezas (como un codo o una rodilla humanos) que permite el movimiento; sin él, el robot sería un bloque rígido.*

b. Tipos de articulaciones

CoppeliaSim ofrece varios tipos, pero los fundamentales en robótica son:

- **Revolute (Rotacional):** Permite el giro alrededor de un eje, funcionando como una bisagra. Es el estándar para la mayoría de brazos industriales.

  > *Funciona igual que un codo o la bisagra de una puerta: solo puede girar, no estirarse.*

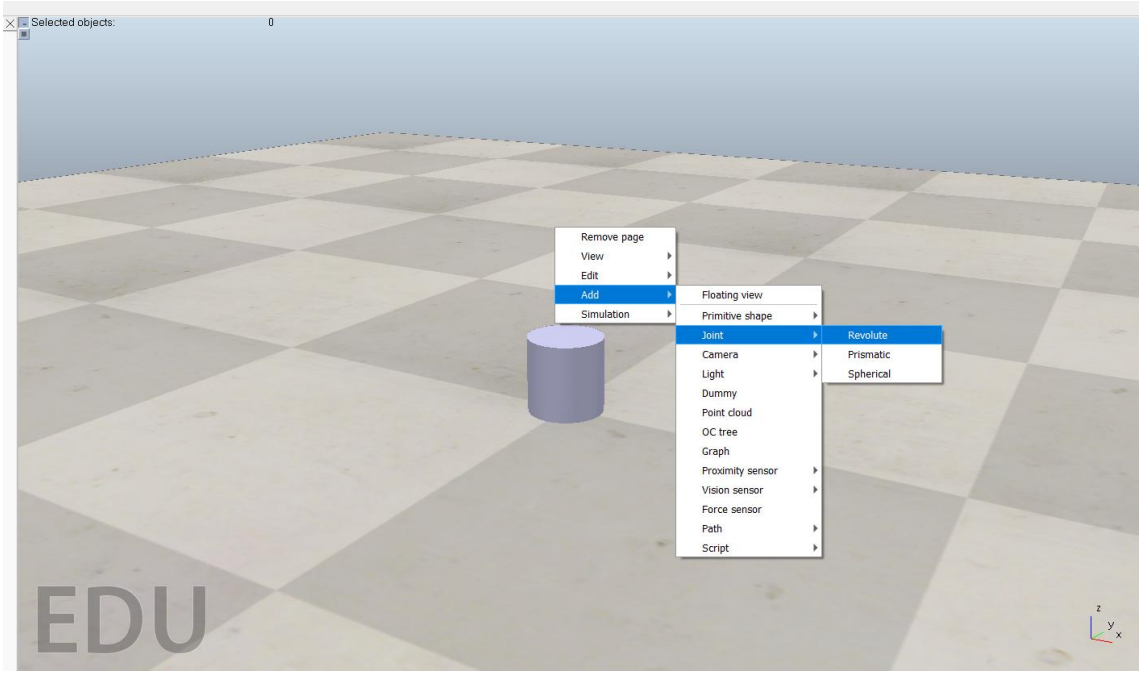
- **Prismatic (Prismática):** Permite un desplazamiento lineal (extensión y contracción) a lo largo de un eje.

  Funciona como un telescopio o un pistón hidráulico: se alarga y se encoge en línea recta.

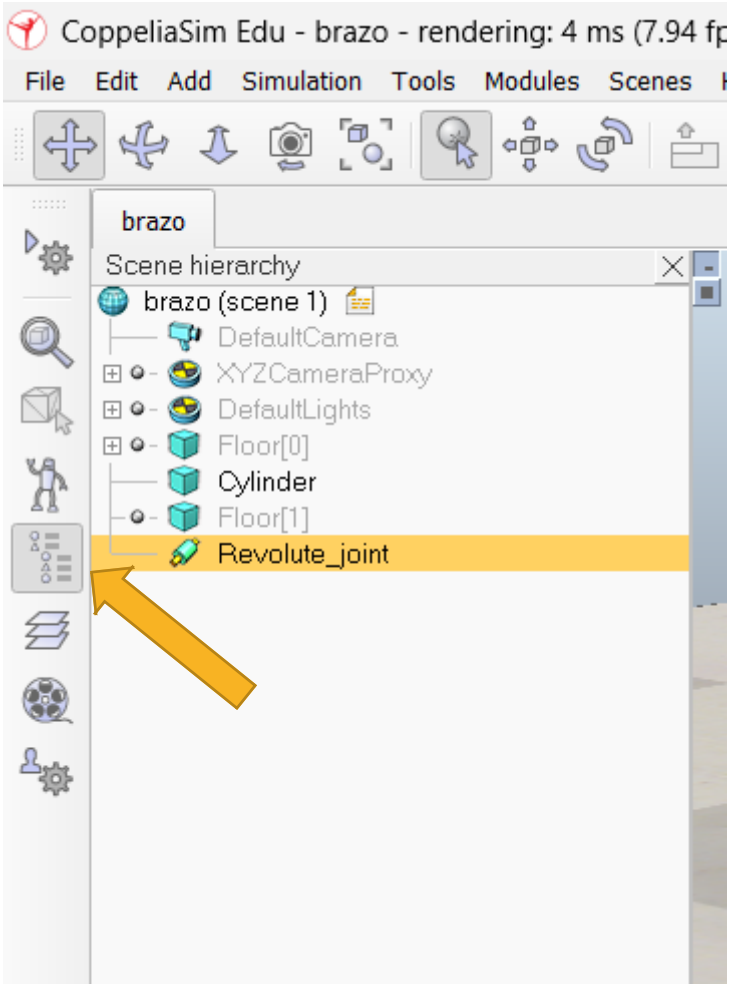
c. Configuración del motor

Para nuestro diseño, utilizaremos una articulación rotacional:

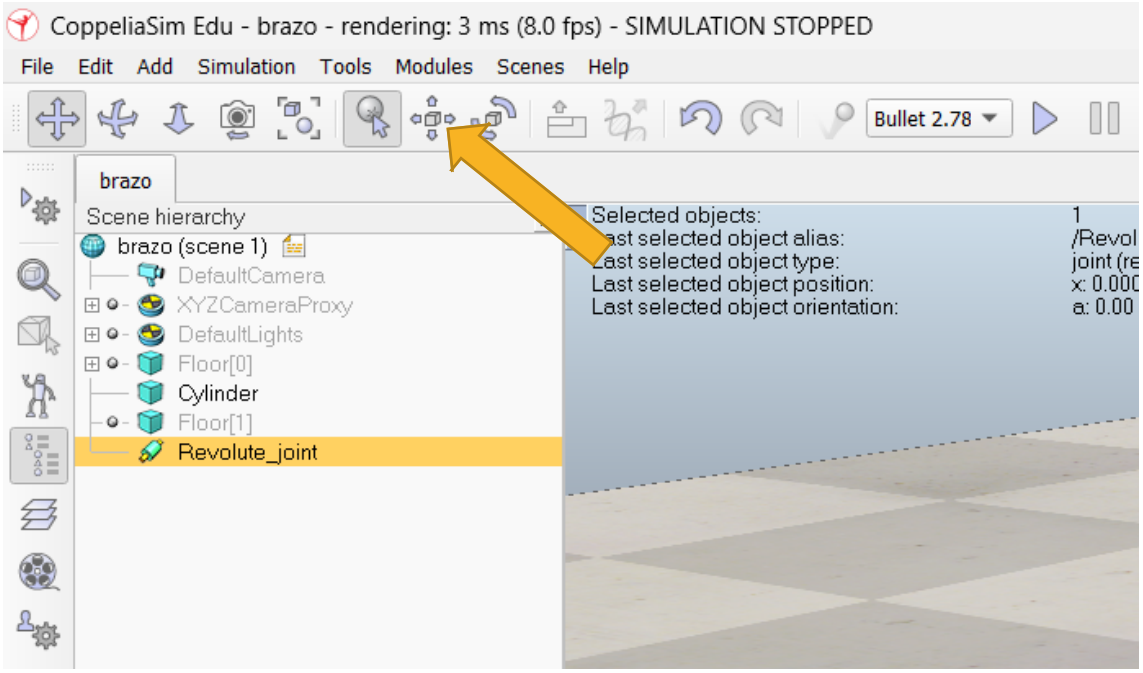
1. Haz clic derecho en la escena: Add > Joint > Revolute.



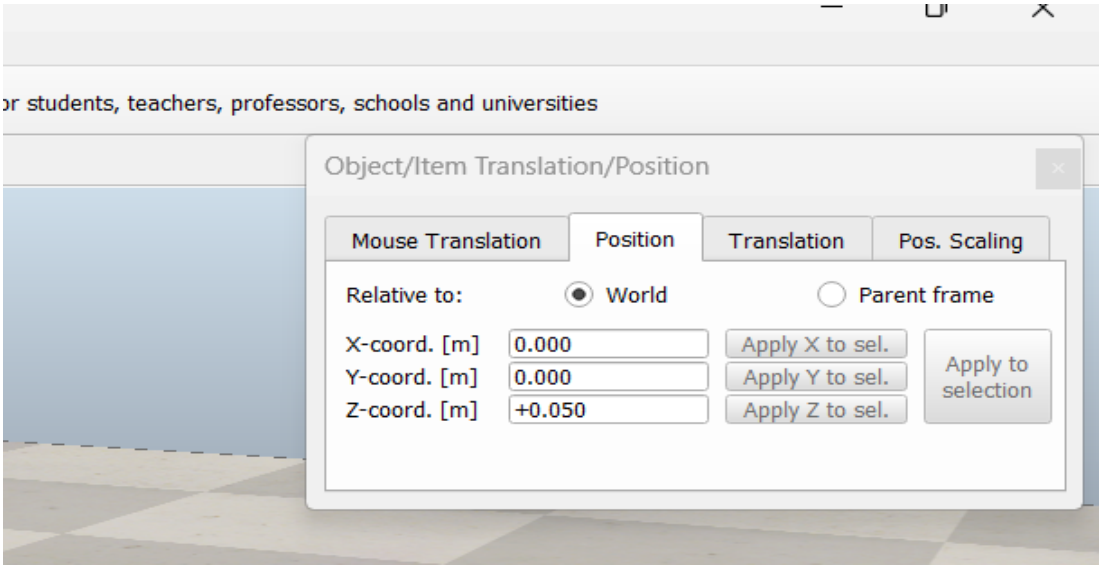
Para visualizar la jerarquía de la escena que hablaremos de ella más adelante debeos seleccionar esta opción en la barra lateral derecha:

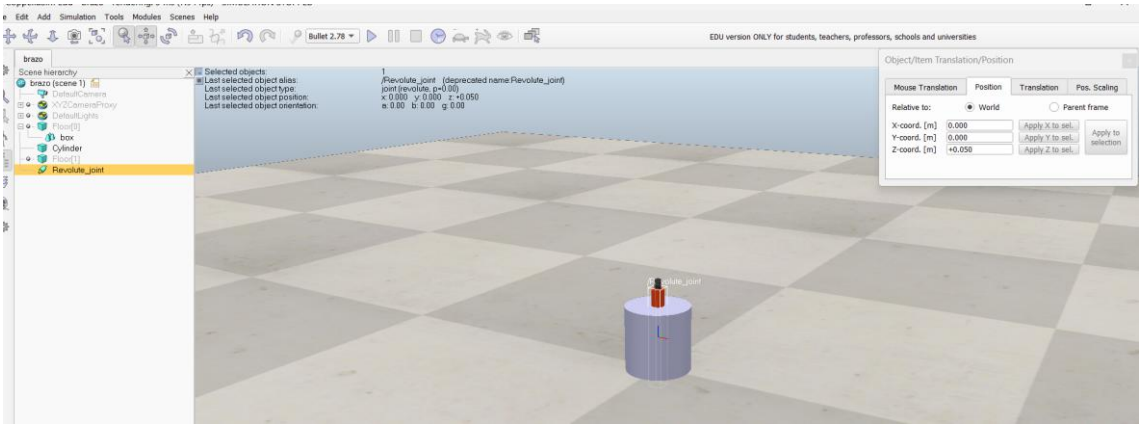


2. **Posicionamiento:** Por defecto, aparece en el suelo (0,0,0) oculta dentro de la base. Para situarla correctamente:
    - Selecciona el objeto Revolute_joint en la jerarquía.
    - Ve a la barra de herramientas superior y pulsa el icono de **Posición** (cuadrado con flechas).



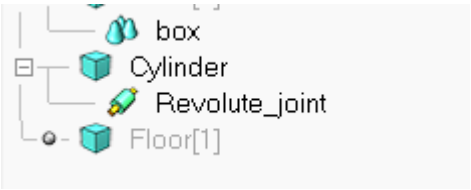
- En la pestaña **Position**, cambia la coordenada **Z-coord** la altura exacta de tu cilindro.





Para que el robot funcione, el motor (Revolute_joint) debe ser "hijo" de la base fija (Cylinder).

1. Dirígete al panel de **"Scene hierarchy"** (situado a la izquierda de la pantalla).
2. Haz clic izquierdo sobre el objeto **Revolute_joint**.
3. Sin soltar el clic, **arrástralo** y suéltalo justo encima del texto **Cylinder**.

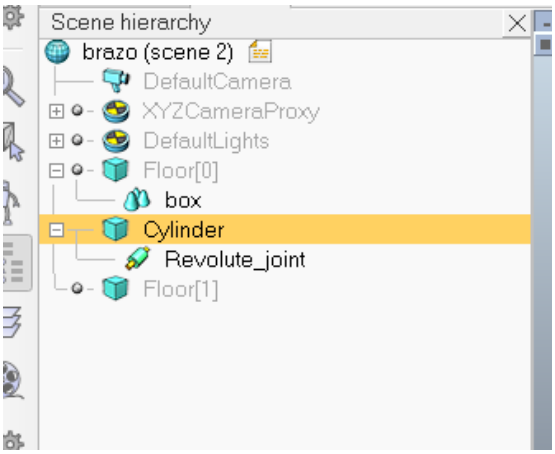


4. Jerarquía y Cinemática (Padre e Hijo)

Para que el robot funcione como una unidad y no como un montón de piezas sueltas, es fundamental entender cómo se agrupan los objetos en el simulador.

a. Qué es el árbol jerárquico

El árbol jerárquico (Scene Hierarchy) es el sistema que utiliza CoppeliaSim para organizar todos los elementos presentes en la simulación. Podremos verlo en el panel lateral izquierdo como una lista. Esta lista no solo muestra los nombres de las piezas, sino que su nivel de "sangría" (desplazamiento hacia la derecha) indica visualmente cómo están conectadas lógicamente entre sí, formando ramificaciones.



Vemos en esta imagen que tenemos la estructura de las piezas que hemos utilizado hasta ahora y las que vienen por defecto.

b. Relación padre-hijo

- **Padre (Parent):** Es el objeto principal o de soporte. Cualquier transformación que sufra (movimiento, rotación o cambio de escala) se transmitirá automáticamente a los elementos que tenga vinculados por debajo.

- **Hijo (Child):** Es el objeto subordinado. Hereda todos los movimientos de su padre. Por ejemplo, si el motor (padre) gira 90 grados, el brazo (hijo) girará esos mismos 90 grados acompañándolo. Sin embargo, si un hijo se mueve de forma independiente, este movimiento no afecta a su padre.

Un ejemplo claro son las piezas que hemos puesto hasta ahora que son el ***Cylinder*** siendo este el padre y ***Revolute_joint*** siendo el hijo

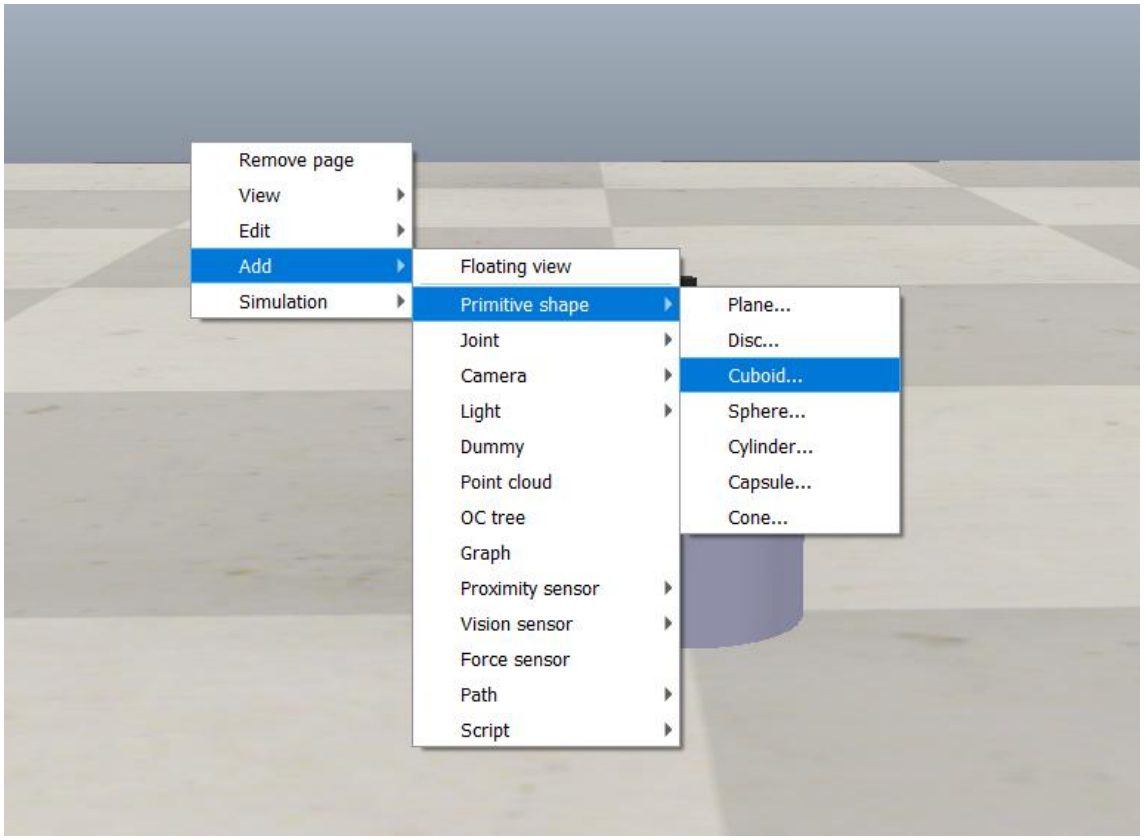
c. Por qué es importante

Establecer la jerarquía es importante para construir una escena sólida. Si los objetos se dejan al mismo nivel, el simulador tratará a esos elementos como independientes, sin estar sujetos entre sí. Es decir, al ejecutar la simulación, las piezas se desplomarían al suelo por la gravedad al no estar ancladas. Por ello es importante establecer esta relación.

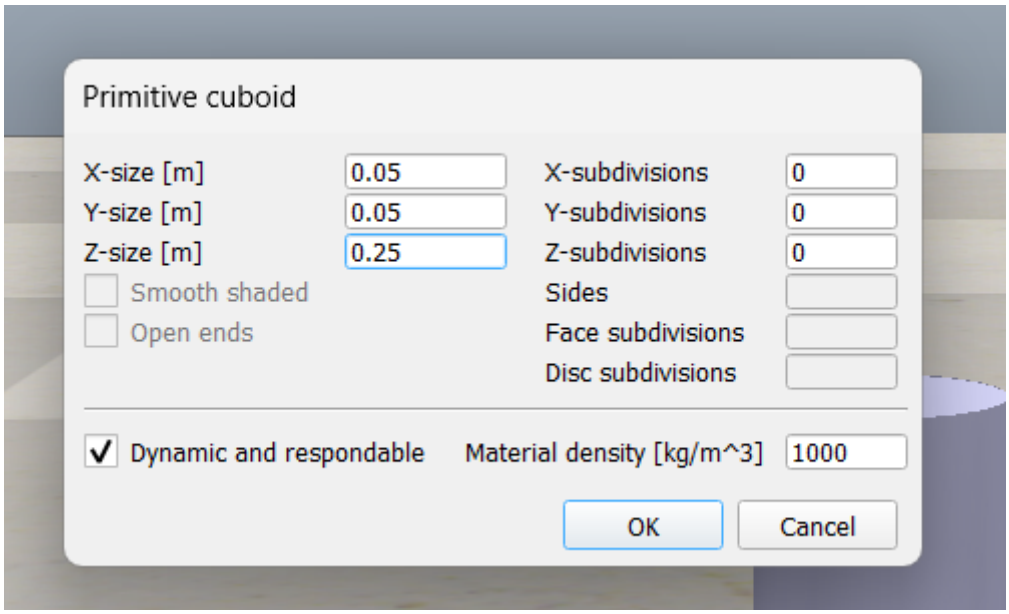
5. Creación del primer eslabón (Brazo 1):

a. Añadir y posicionar la pieza

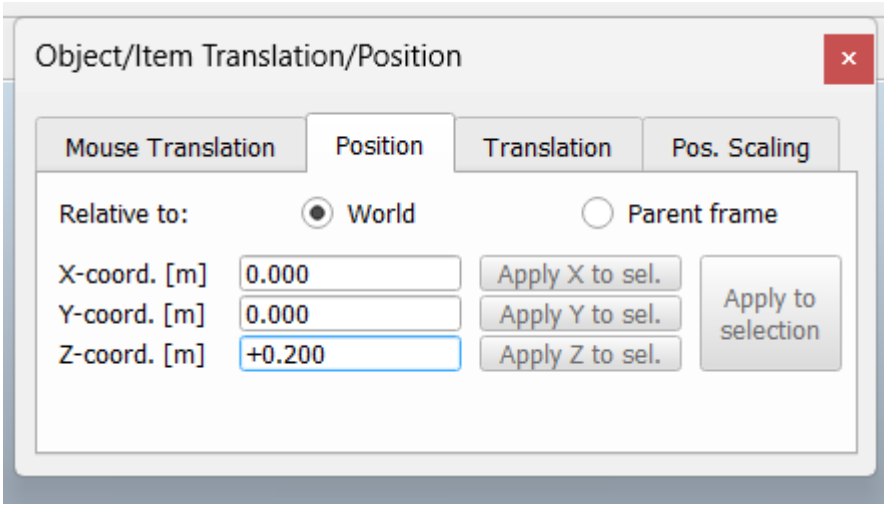
1. Haz clic derecho en el área de trabajo: Add > Primitive shape > Cuboid (Cuboide).



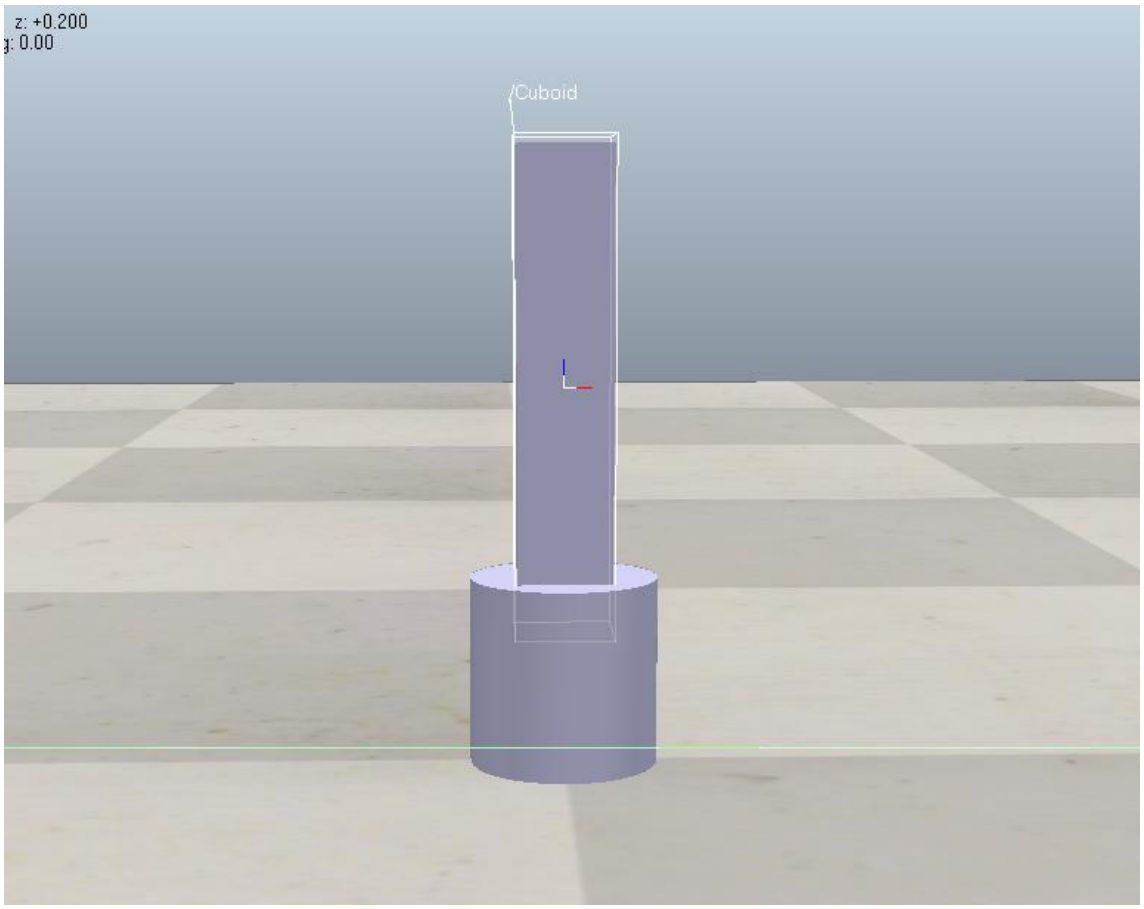
2. Introduce: **X-size:** 0.05, **Y-size:** 0.05, **Z-size:** 0.25. Pulsa OK.



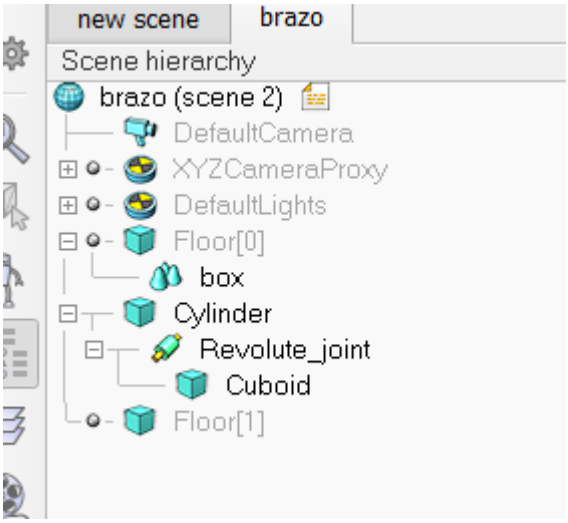
3. Selecciona el Cuboid recién creado, abre la herramienta **Posición** y cambia **Z-coord** a **0.200** para que suba y apoye sobre el motor.



Tendría que verse lo siguiente:



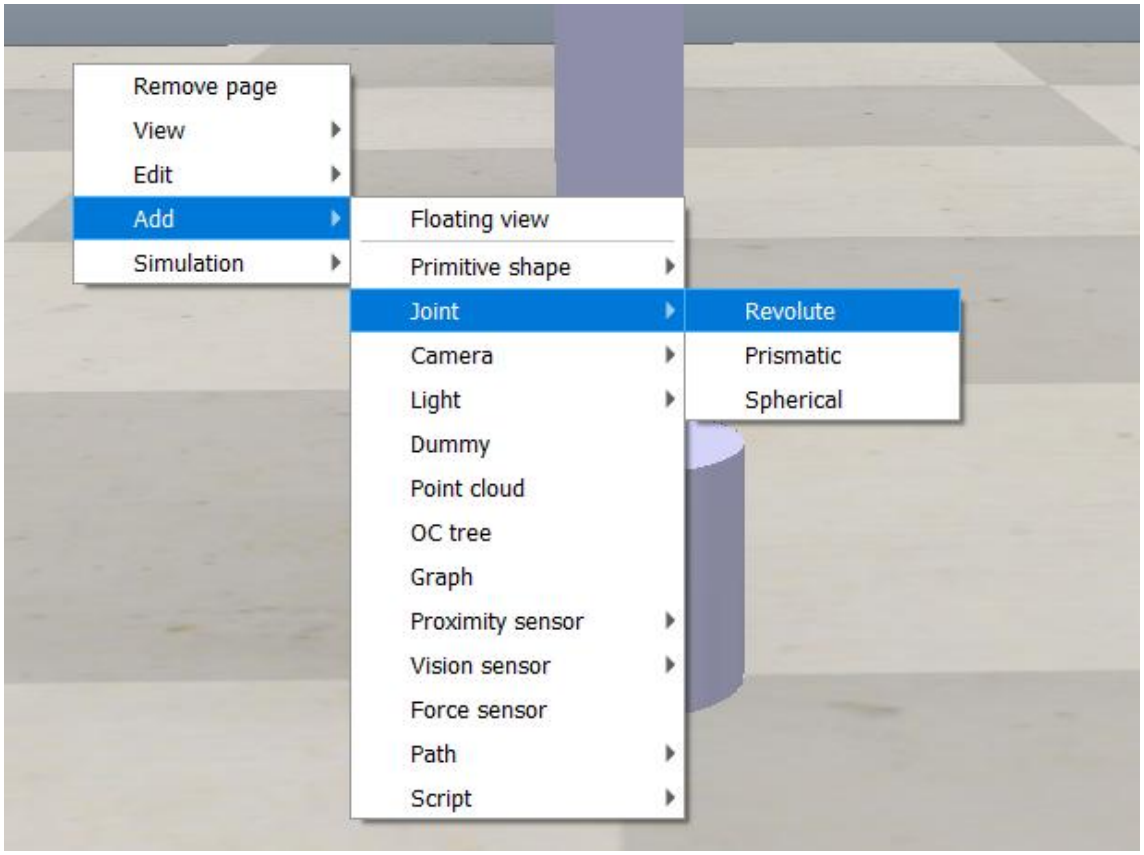
4. Finalmente arrastra el `Cuboid` y suéltalo dentro del `Revolute_joint`.



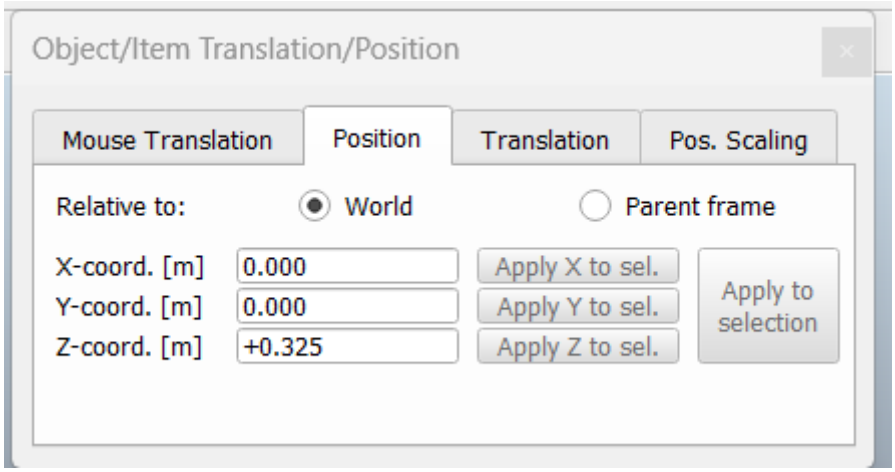
6. Completando el brazo: Codo y Antebrazo

a. Añadir el segundo motor (Codo)

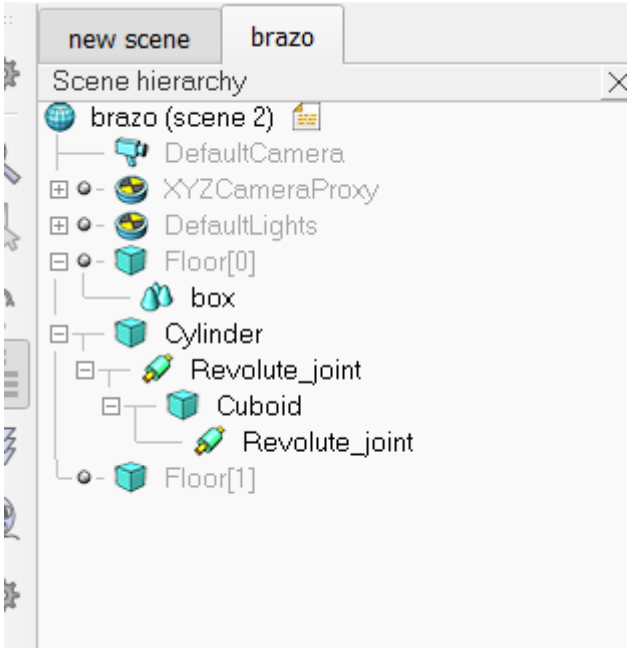
1. Haz clic derecho en la escena: Add > Joint > Revolute.



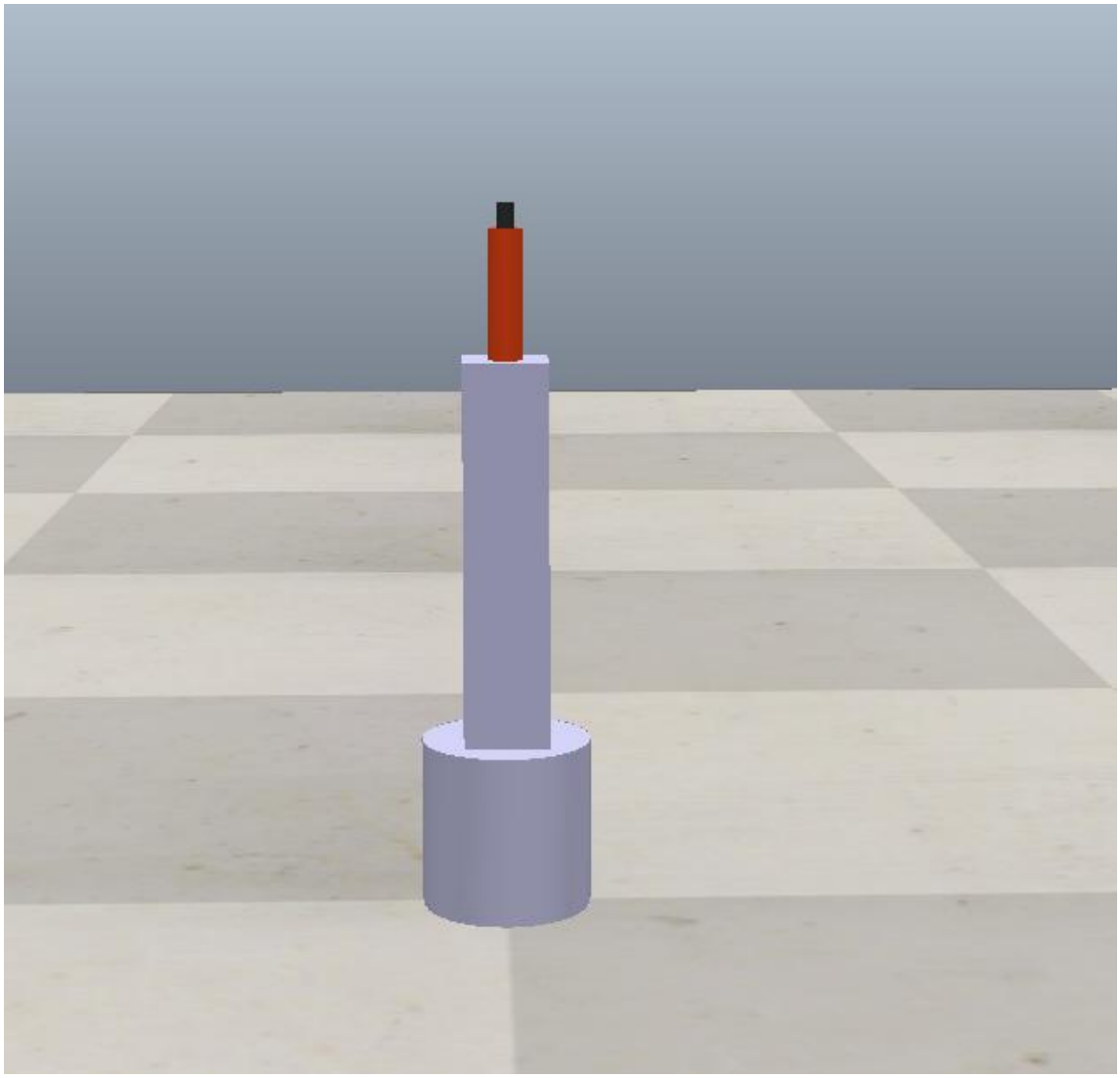
2. Selecciona el nuevo motor (Revolute_joint0) y abre la herramienta de **Posición**.
3. Cambia la coordenada **Z-coord** a **0.325** para que se sitúe justo en la parte superior del primer brazo.



4. **Jerarquía:** Arrastra este nuevo motor y suéltalo **dentro** de Cuboid.



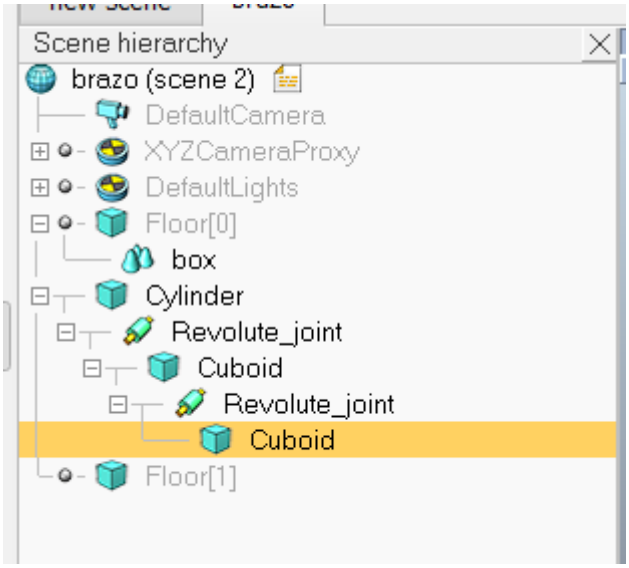
Tendría que verse lo siguiente:

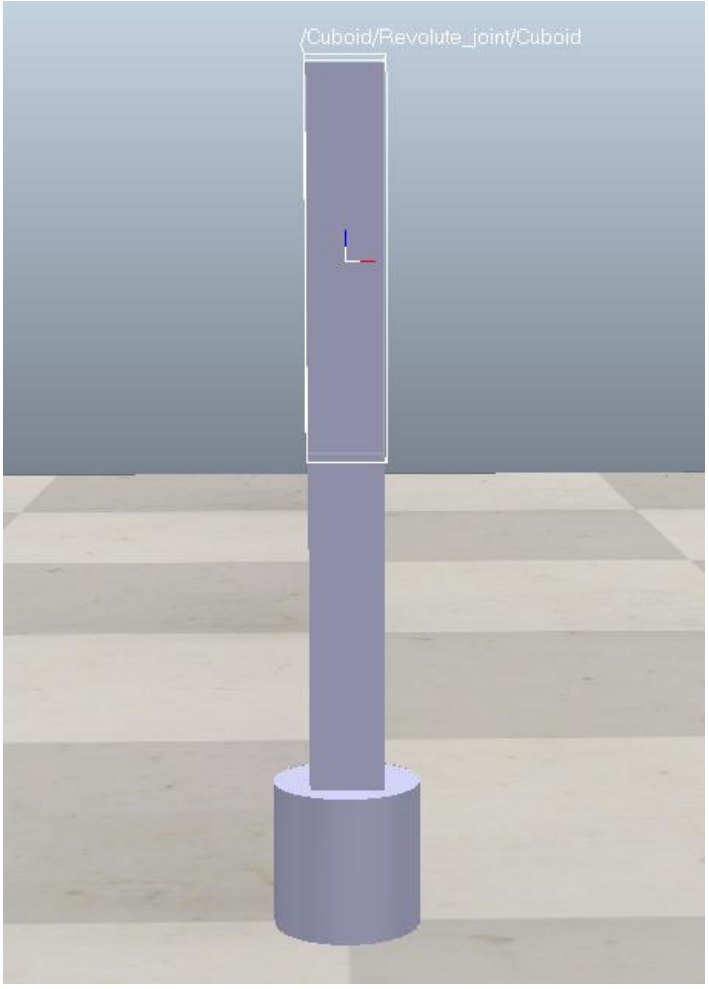


b. Añadir el segundo eslabón (Antebrazo)

1. Haz clic derecho: Add > Primitive shape > Cuboid.
2. Introduce las mismas medidas: X: 0.05, Y: 0.05, Z: 0.25. Pulsa OK.
3. Selecciona el nuevo brazo (Cuboid0), ve a Posición y cambia la Z-coord a 0.450 para que apoye sobre el segundo motor.
4. Jerarquía: Arrastra este nuevo brazo y suéltalo dentro de Revolute_joint0.

Tendría que verse lo siguiente:


Scene hierarchy
brazo (scene 2)
DefaultCamera
XYZCameraProxy
DefaultLights
Floor[0]
box
Cylinder
Revolute_joint
Cuboid
Revolute_joint
Cuboid
Floor[1]


/Cuboid/Revolute_joint/Cuboid

7. Programación y Control (Scripts)

Hasta ahora hemos construido el "esqueleto" y los "músculos" del robot, pero para que cobre vida y se mueva de forma autónoma, necesitamos darle un "cerebro". Esto se consigue mediante la programación de *Scripts*.

a. ¿Qué es un script en CoppeliaSim?

Un script es un bloque de que se asocia a un objeto específico de la jerarquía para controlar su comportamiento durante la simulación. En nuestro caso, crearemos un script asociado a la base del robot para enviarle órdenes de giro a los motores.

b. Diferencia entre scripts Threaded y Non-threaded

En CoppeliaSim, el código se puede ejecutar de dos maneras fundamentales:

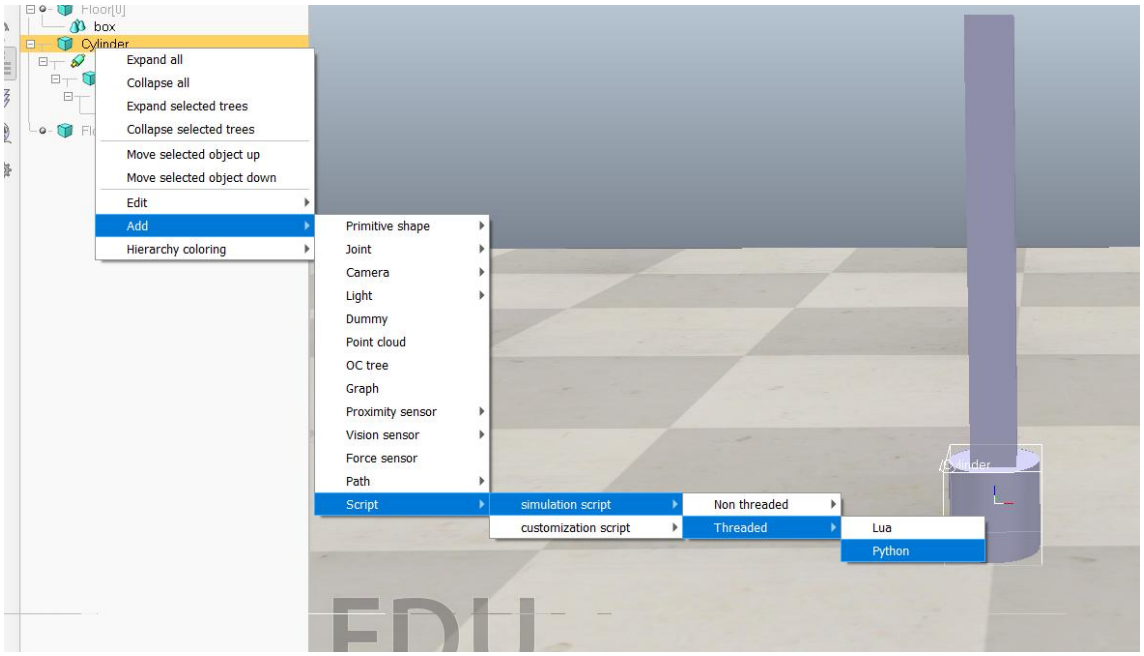
- **Non-threaded (No hilados):** El código se ejecuta de forma secuencial en cada paso de la simulación. Son muy rápidos, pero no puedes usar comandos de "espera", porque si el script se pausa, toda la simulación entera se congela. Se usan para controles continuos en tiempo real.

- **Threaded (Hilados):** Se ejecutan de forma paralela e independiente a la simulación principal (como un hilo separado). Son perfectos para programar rutinas secuenciales o trayectorias (ej. "gira 90 grados, espera 2 segundos y vuelve a bajar"), ya que pueden pausarse sin detener el resto del mundo virtual.

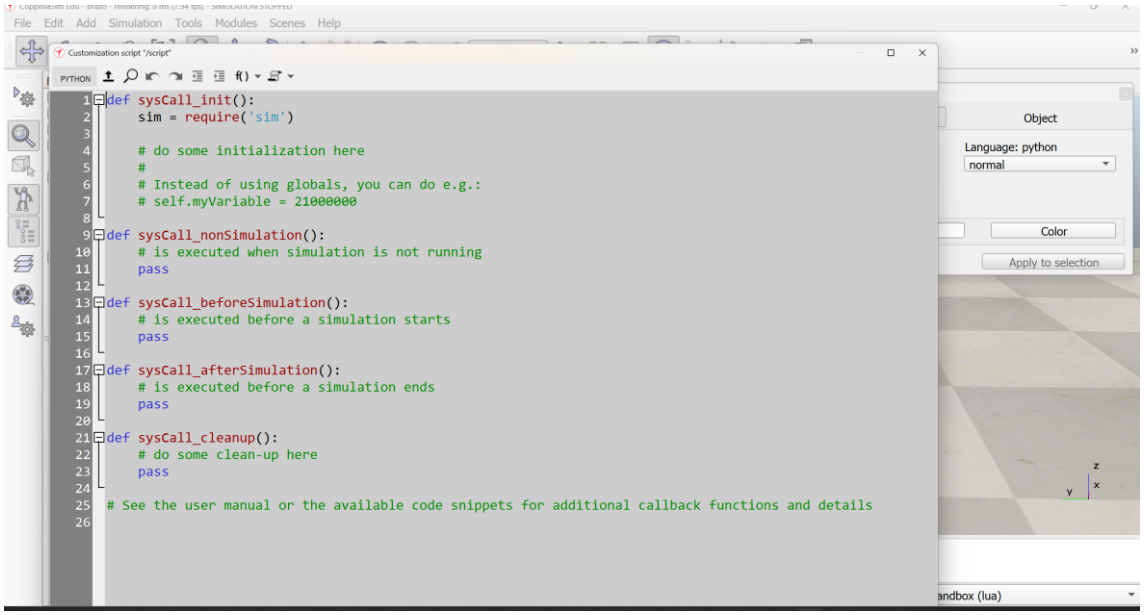
c. Creación del script en la base

Para controlar el brazo, vamos a asociar un script al objeto principal (la base), ya que desde ahí controlaremos a todos sus "hijos":

1. Hacemos clic derecho sobre Cylinder en la jerarquía.
2. Navegamos por: Add > Script > Simulation script > Threaded > Lua.

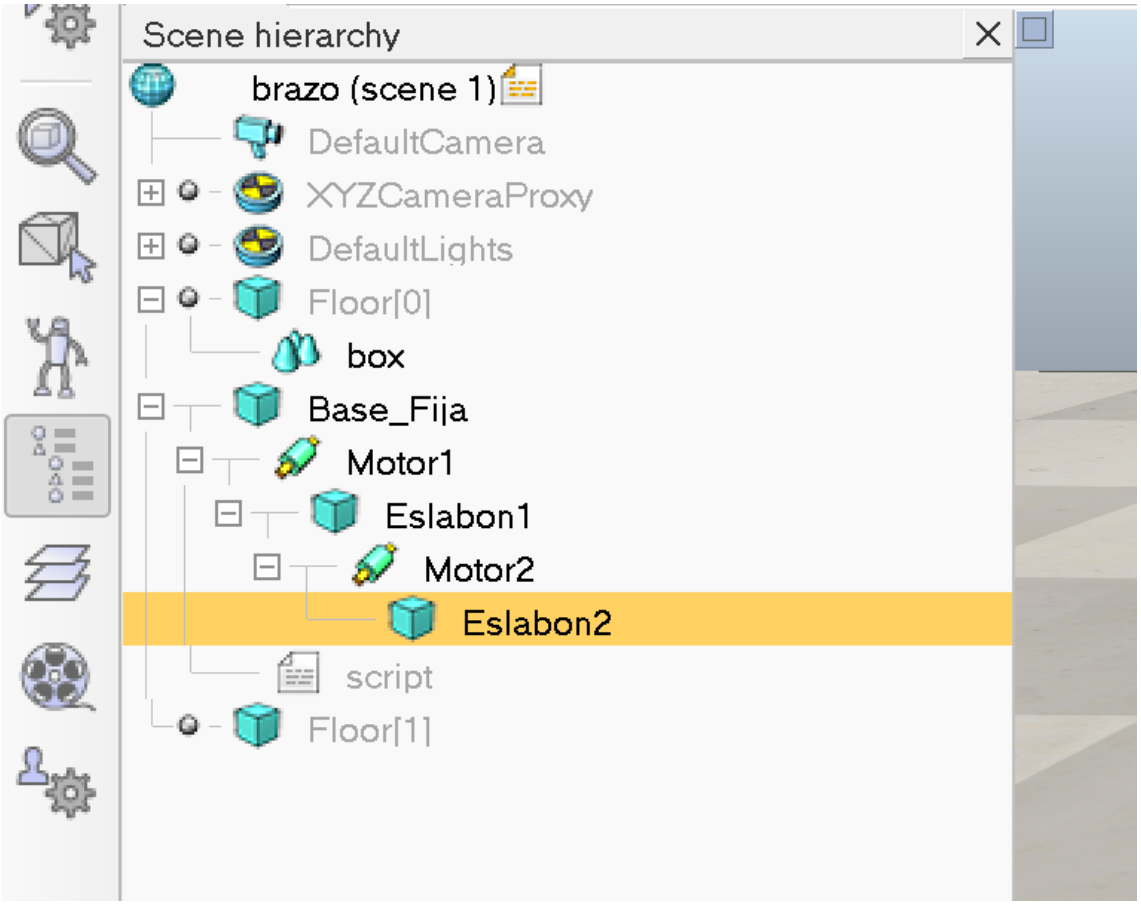


3. Hacemos doble clic en el icono del "pergamino" que aparece junto a Cylinder para abrir el editor de código.



Veremos un código por defecto, seleccionamos todo y lo borramos.

Cambiar nombres de componentes:



d. Código de movimiento automático

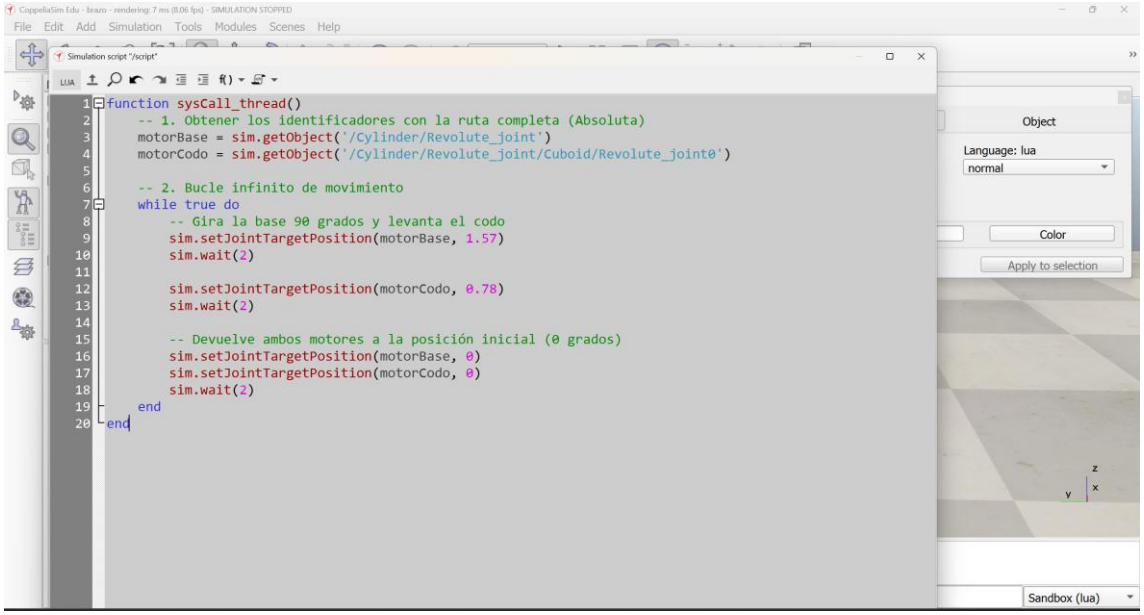
Una vez abierto el editor, borramos el código que viene por defecto y pegamos el siguiente script. Este código creará una secuencia de movimiento continuo para los dos motores:

```
function sysCall_thread()
    -- 1. Obtener los identificadores con la ruta completa (Absoluta)
    local motorBase = sim.getObjectHandle('/Base_Fija/Motor1')
    local motorCodo =
sim.getObjectHandle('/Base_Fija/Motor1/Eslabon1/Motor2')
    -- 2. Bucle infinito de movimiento
    while true do
        sim.setJointTargetPosition(motorBase, 1.57)
        sim.wait(2)

        sim.setJointTargetPosition(motorCodo, 0.78)
```

```
        sim.wait(2)

        -- Devuelve ambos motores a la posici?n inicial (0 grados)
        sim.setJointTargetPosition(motorBase, 0)
        sim.setJointTargetPosition(motorCodo, 0)
        sim.wait(2)
    end
end
```



e. Explicación línea por línea del código

Para entender el "cerebro" de nuestro robot, estas son las funciones clave de forma sencilla:

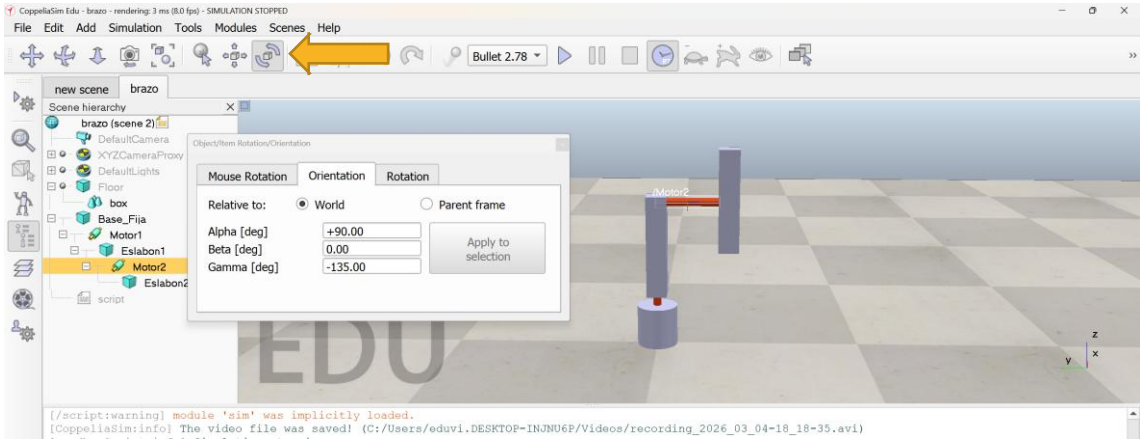
- **function sysCall_thread():** Es la función principal. Al ser *Threaded*, nos permite hacer pausas sin que el simulador entero se quede colgado.
- **sim.getObject(...):** Busca el motor en nuestra jerarquía y lo guarda para que podamos darle órdenes.
- **while true do:** Crea un bucle infinito. El robot repetirá los movimientos sin parar hasta que pulsemos Stop.
- **sim.setJointTargetPosition(...):** Ordena al motor girar a una posición.

- **sim.wait(2):** Hace una pausa de 2 segundos en el código. Esto le da tiempo físico al motor para terminar de moverse antes de recibir la siguiente orden.

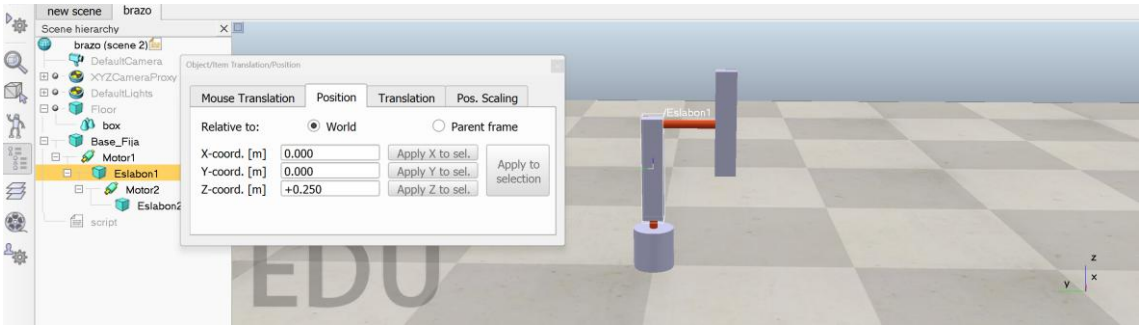
Ajuste de Orientación (Inclinación del Codo)

Pasos para cambiar la orientación del motor:

1. Nos aseguramos de que la simulación está completamente detenida (botón Stop).
2. En el panel de jerarquía de la izquierda, seleccionamos el segundo motor, nombrado en nuestro proyecto como **Motor2**.
3. En la barra de herramientas superior, hacemos clic en la herramienta de **Orientación** (el icono de una esfera rodeada por flechas de rotación).
4. En la ventana emergente, navegamos hasta la pestaña **Orientation**.
5. Modificamos el valor correspondiente al ángulo de inclinación introduciendo **90.00** en la casilla **Alpha** (o Beta, dependiendo de la perspectiva de la cámara).



6. Modificamos la coordenada **Z** a **250** del Eslabon2 para que no roce con la base.



8. Pruebas, Resultados y Resolución de Problemas

Una vez finalizado el ensamblaje, ajustada la orientación de los ejes y programado el script de control, procedemos a verificar el funcionamiento del sistema iniciando la simulación.

a. Resultados de la simulación

Al pulsar el botón "Play", se observa que la jerarquía cinemática funciona correctamente. El Motor1 hace rotar toda la estructura sobre el eje Z, mientras que el Motor2 eleva el Eslabon2 de forma independiente. El bucle infinito programado se ejecuta sin errores, realizando pausas de 2 segundos entre cada instrucción.

FUNCIONAMIENTO DEL BRAZO:

https://drive.google.com/file/d/1-TuhDVnoD2cnSqypy4UC93xPnSL7DgPn/view?usp=sharing

b. Problemas encontrados y soluciones aplicadas

- **Problema:** Al iniciar la simulación por primera vez, el robot completo se desplomaba y caía al suelo. Esto ocurría porque el centro de gravedad del conjunto se desplazaba y el motor principal no estaba anclado al entorno.

- **Solución:** Se seleccionó la base del robot (Base_Fija), se abrió el menú *Dynamic properties dialog* y se desmarcó la casilla **"Body is dynamic"**. Al convertir la base en un objeto estático (no afectado por la gravedad, pero capaz de colisionar), la estructura quedó sólidamente anclada al suelo virtual, soportando perfectamente el movimiento de los eslabones superiores.